

Byte Bazaar Project Guide

A simple explanation of the full project, with extra frontend and AngularJS notes.

1. Project Overview

Byte Bazaar is a small electronics shopping website. It is made as a simple full-stack project using Spring Boot for the backend and AngularJS with HTML/CSS for the frontend.

The main idea is simple: users can register, login, see products, add products to cart, place an order, and view previous orders. The project uses H2 database by default, so it runs without installing MySQL. MySQL support is also present through a separate Spring profile.

Main Features

- User registration and login.
- Product catalog with search and category filter.
- Cart add, update, and remove options.
- Checkout that creates an order and reduces stock.
- Order history for logged-in users.

2. Technology Used

Part	Technology	Use
Backend	Java, Spring Boot	Runs API endpoints and business logic.
Database	H2 / MySQL	Stores users, products, cart items, tokens, and orders.
Frontend	HTML, CSS, AngularJS	Shows pages and calls backend APIs.
Build Tool	Gradle	Runs, builds, and tests the project.

3. Folder Structure

```
ByteBazaar
|-- build.gradle
|-- README.md
|-- docs
|   |-- ByteBazaar_Project_Guide.html
|   |-- ByteBazaar_Project_Guide.pdf
|-- src
|   |-- main
|       |-- java/org/varun/bytebazaar
|       |   |-- auth
|       |   |-- cart
```

```
| | | |-- catalog
| | | |-- common
| | | |-- config
| | | |-- orders
| | | |-- users
| | |-- resources
| | |-- static
| | | |-- index.html
| | | |-- styles.css
| | | |-- app.js
| | |-- application.properties
| | |-- application-mysql.properties
| |-- test
```

4. Backend Explanation

The backend is split into packages based on project features. This keeps the code easier to understand.

auth package

This package handles registration, login, logout, and current-user API. When a user logs in, the backend creates a simple token and saves it in the database. The frontend sends this token in the Authorization header for protected APIs.

users package

This package has the user entity, role enum, and repository. User data like name, email, password hash, and role is stored here.

catalog package

This package has product entity, repository, and product controller. Product APIs are public, so users can view products without login.

cart package

This package handles cart APIs. Cart items are connected to a user and a product. The service checks product stock before adding or updating cart quantity.

orders package

This package handles checkout and order history. During checkout, the cart is converted into an order. Product details are copied into order items so the order history remains correct even if product data changes later.

config package

This package contains security setup, token filter, and data seeding. The data seeder inserts demo products and a demo user when the database is empty.

5. How Request Flow Works

Example: adding a product to cart.

1. User clicks Add to cart button in frontend.
2. app.js sends POST /api/cart request with product id and quantity.
3. Token is sent in Authorization header.
4. TokenAuthenticationFilter checks token and finds logged-in user.
5. CartController receives request.
6. CartService checks product and stock, then saves cart item.
7. Backend returns updated cart.
8. Frontend updates cart count and total.

6. How to Run the Project

```
.\gradlew.bat bootRun
```

Open this URL in browser:

```
http://localhost:8080
```

Demo login:

```
Email: customer@bytebazaar.dev  
Password: password
```

7. Frontend Files

The frontend is inside:

```
src/main/resources/static
```

File	Purpose
index.html	Main page structure. Contains sections for catalog, account, cart, and orders.
styles.css	Controls layout, colors, spacing, cards, buttons, and mobile view.
app.js	AngularJS code. Stores frontend data and calls backend APIs.

8. AngularJS Basic Idea

AngularJS is a JavaScript framework that connects HTML with JavaScript data. In normal HTML, if data changes in JavaScript, we manually update the page using DOM methods. In AngularJS, we connect HTML elements to JavaScript variables using special attributes like `ng-model`, `ng-click`, `ng-repeat`, and `ng-show`.

In this project, AngularJS is used only for simple work: showing data, handling button clicks, forms, and API calls. There is no complex frontend architecture.

AngularJS App Start

```
<html lang="en" ng-app="byteBazaarApp">
```

`ng-app` tells AngularJS that this HTML page belongs to the app named `byteBazaarApp`. The same name is used in `app.js`:

```
angular.module('byteBazaarApp', [])
```

Controller

```
<body ng-controller="StoreController as store">
```

The controller is the JavaScript part that controls the page. Here, `StoreController` is used as `store` in HTML. That is why the HTML has code like:

```
store.products
store.cart
store.show('cart')
store.login()
```

9. AngularJS Directives Used

Directive	Meaning	Example in Project
ng-app	Starts AngularJS app on the page.	ng-app="byteBazaarApp"
ng-controller	Connects a controller to HTML.	StoreController as store
ng-click	Runs a JavaScript function when clicked.	ng-click="store.addToCart(product)"
ng-model	Connects input value to JavaScript variable.	ng-model="store.search"
ng-repeat	Loops over an array and repeats HTML.	ng-repeat="product in store.products"
ng-if	Adds/removes HTML based on condition.	ng-if="store.user"
ng-show	Shows/hides HTML based on condition.	ng-show="store.view === 'catalog'"
ng-submit	Runs function when form is submitted.	ng-submit="store.login()"

10. Understanding index.html

Header

The header contains the site name and navigation buttons. Buttons do not move to separate pages. Instead, they call `store.show(...)` and change `store.view`.

```
<button ng-click="store.show('catalog')">Catalog</button>
<button ng-click="store.show('cart')">Cart</button>
```

Messages

Success and error messages are shown using `ng-if`. If `store.message` is empty, the success box is not shown.

```
<section class="notice" ng-if="store.message">{{store.message}}</section>
```

Catalog

The catalog uses `ng-repeat` to show every product. The filter calls `store.productFilter` so search and category selection work.

```
<article ng-repeat="product in store.products | filter:store.productFilter">
```

Forms

Login and register forms use `ng-model`. This means when the user types in an input, AngularJS stores that value inside JavaScript object like `store.loginForm.email`.

```
<input type="email" ng-model="store.loginForm.email">
```

11. Understanding app.js

app.js is the main frontend logic file. It stores page data and defines functions used by buttons and forms.

View Model

```
var vm = this;
```

vm means view model. It is a common AngularJS style. Anything added to vm can be used in HTML through store.

Page State

```
vm.view = 'catalog';  
vm.products = [];  
vm.cart = {items: [], total: 0};
```

These variables store the current screen and data. When values change, AngularJS updates the HTML automatically.

API Calls With \$http

\$http is AngularJS service for calling backend APIs. Example:

```
$http.get('/api/products').then(function (response) {  
    vm.products = response.data;  
});
```

Here, frontend calls /api/products. When backend sends response, product data is stored in vm.products. The catalog updates automatically.

Local Storage

localStorage is browser storage. This project stores token and user data there, so refreshing the page does not immediately log the user out.

```
localStorage.setItem('byteBazaarToken', vm.token);  
localStorage.getItem('byteBazaarToken');
```

12. Understanding styles.css

CSS controls how the frontend looks. This project uses simple CSS without Bootstrap or Tailwind.

Grid Layout

```
.product-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 16px;
}
```

This creates product cards in columns. `auto-fit` adjusts the number of columns based on screen width. `minmax(250px, 1fr)` means each card should be at least 250px wide, and available space is shared.

Flex Layout

```
.topbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

Flexbox is used for simple row layouts like header and price/stock rows.

Media Query

```
@media (max-width: 760px) {
  .topbar,
  .toolbar,
  .auth-panel,
  .line-item {
    display: grid;
    grid-template-columns: 1fr;
  }
}
```

This makes the layout mobile-friendly. When screen width is below 760px, sections are stacked vertically.

13. Frontend Flow Examples

Login Flow

1. User enters email and password.
2. Form calls `store.login()`.
3. `app.js` sends `POST /api/auth/login`.
4. Backend returns token and user details.
5. Frontend saves token in `localStorage`.
6. Frontend adds token to future API headers.

Add to Cart Flow

1. User clicks Add to cart.
2. Button calls `store.addToCart(product)`.

3. Frontend sends product id and quantity to backend.
4. Backend returns updated cart.
5. Frontend replaces old cart data with new cart data.

Checkout Flow

1. User enters shipping address.
2. Form calls `store.checkout()`.
3. Backend creates order from cart.
4. Backend clears cart and reduces stock.
5. Frontend clears cart on screen and opens orders view.

14. Important API Endpoints

Endpoint	Use
POST /api/auth/register	Create new user.
POST /api/auth/login	Login and get token.
GET /api/products	Get products for catalog.
GET /api/cart	Get logged-in user's cart.
POST /api/cart	Add product to cart.
PATCH /api/cart/{itemId}	Update cart item quantity.
DELETE /api/cart/{itemId}	Remove cart item.
POST /api/orders	Place order.
GET /api/orders	View previous orders.

15. How I Made/Simplified It

The project was kept simple so it looks like a realistic college-level full-stack project. The backend was divided feature-wise instead of creating very complex architecture. The token login was simplified using a database-stored UUID token. The UI was made with plain HTML, CSS, and AngularJS instead of using a large frontend framework.

Product data was also changed to simple real-world names and short descriptions. The README was rewritten in plain language. Later, comments were added in frontend and important backend classes so the code is easier to explain.

16. What to Study First

1. Open `index.html` and understand the screens.
2. Open `app.js` and match each button/form with its function.
3. Open `styles.css` and see how classes from HTML are styled.

4. Then study backend controllers, because controller methods match API calls from frontend.
5. After controllers, study services because actual business logic is inside services.

Tip: For frontend understanding, do not start by memorizing AngularJS. First match HTML attributes like `ng-click` and `ng-model` with functions and variables in `app.js`. That will make the project easier to understand.